

Technical Analysis: Password Leak Checker & Generator

System Architecture Documentation

March 23, 2026

1 Introduction

This document provides a comprehensive analysis of the `Leak Checker` application. The application is designed to evaluate password strength, check for known data breaches using the HaveIBeenPwned (HIBP) API via k-anonymity, and generate cryptographically secure passwords.

2 Core Principles

2.1 K-Anonymity via SHA-1

The primary security feature of this application is the implementation of k-anonymity for checking breached passwords.

1. **Hashing:** The password is hashed using SHA-1.
2. **Prefixing:** Only the first 5 characters (20 bits) of the 40-character hex string are sent to the HIBP API.
3. **Privacy:** The API returns a list of all breached suffixes starting with that prefix. The server never sees the full hash, ensuring the password remains private even if the API provider were malicious.

2.2 Information Entropy

The application calculates the theoretical strength of a password using entropy, measured in bits. The formula used is:

$$E = L \times \log_2(R)$$

Where:

- E is the entropy in bits.
- L is the length of the password.
- R is the size of the character pool (range) used (e.g., 26 for lowercase, 10 for digits).

3 Documented Source Code

The following is the fully annotated Python implementation using the Flask framework.

```

1 import csv, hashlib, hmac, math, os, re, secrets, string, urllib.request, urllib
  .error
2 from flask import Flask, request, jsonify, send_from_directory
3
4 # Initialize Flask application
5 app = Flask(__name__, static_folder="static")
6
7 # --- Common Password Module ---
8 # This section handles checking against a local list of common/dictionary
  passwords.
9
10 def load_common(path):
11     """Loads a CSV file containing common passwords into a memory-efficient set.
12     """
13     if not os.path.isfile(path):
14         return set()
15     with open(path, newline="", encoding="utf-8", errors="ignore") as f:
16         # We strip whitespace and only add non-empty rows
17         return {row[0].strip() for row in csv.reader(f) if row}
18
19 # Resolve the absolute path to the common passwords file
20 _here = os.path.dirname(os.path.abspath(__file__))
21 COMMON = load_common(os.path.join(_here, "commonpwds.csv"))
22
23 def is_common(pw):
24     """Checks if the password exists in the common list (case-insensitive)."""
25     return pw in COMMON or pw.lower() in COMMON
26
27 # --- Strength Evaluation Module ---
28 # Evaluates passwords based on length, variety, and patterns.
29
30 def strength(pw):
31     """Performs a multi-factor analysis of password strength."""
32     n = len(pw)
33
34     # Check for character types using Regular Expressions
35     has = {
36         "upper": bool(re.search(r'[A-Z]', pw)),
37         "lower": bool(re.search(r'[a-z]', pw)),
38         "digit": bool(re.search(r'\d', pw)),
39         "symbol": bool(re.search(r'[!@#$%^&*~.-_`|{}~]', pw)),
40     }
41
42     # Base score calculated from length
43     score = (3 if n >= 16 else 2 if n >= 12 else 1 if n >= 8 else 0)
44     # Add points for each character type variety
45     score += sum(has.values())
46
47     # Penalty: Repeated characters (e.g., 'aaaa')
48     if re.search(r'(\d|a-z|A-Z){2,}', pw): score -= 1
49     # Penalty: Sequential patterns (e.g., '123' or 'abc')
50     if re.search(r'(012|123|234|345|456|567|678|789|abc|bcd|def|ghi|jkl|mno|pqr|rst|uvw|xyz)', pw.lower()):
51         score -= 1
52
53     # Clamp score between 0 and 10
54     score = max(0, min(score, 10))
55
56     # Entropy calculation

```

```

55     # Define pool size based on types present
56     pool = sum([has["lower"]*26, has["upper"]*26, has["digit"]*10, has["symbol"
]*32]) or 1
57     entropy = round(n * math.log2(pool))
58
59     # Map score to user-friendly label
60     label = "strong" if score >= 8 else "moderate" if score >= 5 else "weak"
61
62     # Generate actionable tips for the user
63     tips = []
64     if n < 12: tips.append("use 12+ characters")
65     if not has["upper"]: tips.append("add uppercase letters")
66     if not has["lower"]: tips.append("add lowercase letters")
67     if not has["digit"]: tips.append("add numbers")
68     if not has["symbol"]: tips.append("add symbols (!@#$...)")
69     if re.search(r'(\.)\1{2,}', pw): tips.append("avoid repeated characters")
70     if re.search(r'(012|123|234|345|456|567|678|789|abc|bcd)', pw.lower()): tips
.append("avoid sequential patterns")
71
72     return {
73         "score": score,
74         "label": label,
75         "entropy": entropy,
76         "tips": tips,
77         "checks": has,
78         "length": n
79     }
80
81 # --- HIBP Breach Check Module ---
82 # Implements the k-anonymity protocol for leak checking.
83
84 def check_hibp(pw):
85     """Checks the password against the HaveIBeenPwned API securely."""
86     # 1. Generate SHA-1 hash
87     sha1 = hashlib.sha1(pw.encode()).hexdigest().upper()
88     # 2. Split into 5-char prefix and the rest (suffix)
89     prefix, suffix = sha1[:5], sha1[5:]
90
91     # 3. Request suffixes starting with the prefix from the API
92     req = urllib.request.Request(
93         f"https://api.pwnedpasswords.com/range/{prefix}",
94         headers={"User-Agent": "passguard/2.0"}
95     )
96
97     try:
98         with urllib.request.urlopen(req, timeout=5) as resp:
99             # 4. Iterate through returned suffixes
100             for line in resp.read().decode().splitlines():
101                 # Format is "SUFFIX:COUNT"
102                 h, n = line.split(":")
103                 # 5. Use constant-time comparison to prevent timing attacks
104                 if hmac.compare_digest(h, suffix):
105                     return int(n) # Found! Return breach count
106             return 0 # Not found in any recorded breach
107     except Exception:
108         return None # Indicate a network or API error
109
110 # --- Secure Generator Module ---

```

```

111 # Uses the 'secrets' module for cryptographically strong randomness.
112
113 WORDS = [
114     "amber", "blaze", "cedar", "delta", "ember", "frost", "gleam", "haven",
115     "ivory", "jumbo", "karma", "lunar", "maple", "nexus", "orbit", "polar",
116     "quartz", "river", "storm", "twist", "ultra", "vault", "whale", "xenon",
117     "yacht", "zesty", "brave", "coral", "dream", "eagle", "flame", "grape",
118 ]
119
120 def gen_password(length=20, upper=True, digits=True, symbols=True, no_ambiguous=
False):
121     """Generates a random character-based password."""
122     pool = string.ascii_lowercase
123     if upper: pool += string.ascii_uppercase
124     if digits: pool += string.digits
125     if symbols: pool += "!@#$%^&*()-_+= "
126
127     # Remove confusing characters like 'l', '1', '0', 'O' if requested
128     if no_ambiguous: pool = pool.translate(str.maketrans("", "", "0011I"))
129
130     # Ensure at least one of each required type is included initially
131     required = [secrets.choice(string.ascii_lowercase)]
132     if upper: required.append(secrets.choice(string.ascii_uppercase))
133     if digits: required.append(secrets.choice(string.digits))
134     if symbols: required.append(secrets.choice("!@#$%^&*"))
135
136     # Fill the remaining length with random choices from the pool
137     rest = [secrets.choice(pool) for _ in range(length - len(required))]
138     combined = required + rest
139
140     # Shuffle the list to hide the fixed positions of 'required' chars
141     secrets.SystemRandom().shuffle(combined)
142     return "".join(combined)
143
144 def gen_passphrase(n=4):
145     """Generates a human-readable but high-entropy passphrase."""
146     words = [secrets.choice(WORDS) for _ in range(n)]
147     sep = secrets.choice(["-", ".", "_"])
148     # Append a small random number and symbol to increase complexity
149     return sep.join(words) + str(secrets.randbelow(900) + 100) + secrets.choice(
"!@#$")
150
151 # --- API Route Handlers ---
152
153 @app.route("/")
154 def index():
155     """Serves the frontend interface."""
156     return send_from_directory("static", "index.html")
157
158 @app.route("/api/check", methods=["POST"])
159 def api_check():
160     """Endpoint to evaluate a password provided by the user."""
161     pw = request.json.get("password", "")
162     if not pw:
163         return jsonify({"error": "no password provided"}), 400
164
165     # Execute all checks
166     s = strength(pw)

```

```

167     common = is_common(pw) if COMMON else None
168     hibp = check_hibp(pw)
169
170     return jsonify({
171         "strength": s,
172         "common": common,
173         "common_list_size": len(COMMON),
174         "hibp_count": hibp,
175     })
176
177 @app.route("/api/generate", methods=["POST"])
178 def api_generate():
179     """Endpoint to generate secure passwords or passphrases."""
180     data = request.json or {}
181     mode = data.get("mode", "random")
182
183     if mode == "passphrase":
184         n = int(data.get("words", 4))
185         results = [gen_passphrase(n) for _ in range(5)]
186     else:
187         # Default character-based generation
188         length = int(data.get("length", 20))
189         upper = bool(data.get("upper", True))
190         digits = bool(data.get("digits", True))
191         symbols = bool(data.get("symbols", True))
192         no_ambig = bool(data.get("no_ambiguous", False))
193         count = int(data.get("count", 5))
194         results = [gen_password(length, upper, digits, symbols, no_ambig) for _
195                     in range(count)]
196
197     return jsonify({
198         "passwords": [{"value": p, "strength": strength(p)} for p in results]
199     })
200 if __name__ == "__main__":
201     # Start the server on port 3030
202     port = int(os.environ.get("PORT", 3030))
203     app.run(host="0.0.0.0", port=port)

```

Listing 1: Annotated Flask Backend Implementation

4 Recommendations for Production

While this code is robust, consider the following for a production-grade deployment:

- **HTTPS Requirement:** Never transmit raw passwords over plain HTTP. Ensure SSL/TLS is enabled.
- **Client-Side Hashing:** To achieve the goal of “password never leaves the machine,” move the SHA-1 hashing logic to the browser using the Web Crypto API.
- **Rate Limiting:** Implement rate limiting on the `/api/check` endpoint to prevent users from testing thousands of passwords (which could facilitate a “checking-as-a-service” for attackers).